```
    if(_cache.TryGetValue<List<string>>(
    ➥ nameof(GetCityNamesAsync), out var cityNames))      ❶
    {
        _logger.LogInformation(
        ➥ "Cities retrieved from memory cache");          ❷
        return cityNames;                                  ❷
    }
    _cache.Set(nameof(GetCityNamesAsync),
    ➥ await _context.Cities.Select(c =>
    ➥ c.Name.ToLowerInvariant()).ToListAsync));           ❸
    _logger.LogInformation("Cache empty.                   ❸
    ➥ Storing cities in memory cache");                   ❸
    return _cache.Get<List<string>>(                       ❸
    ➥ nameof(GetCityNamesAsync));                         ❸
}
```

❶ Use the TryGetValue method to see if a cache entry exists.

❷ If it exists, log a message and return the out parameter.

❸ Otherwise, populate the cache from the database, log a different message, and then return the cache contents.

The `Set` method has a number of overloads that enable you to set an expiry on the item, similar to the expiry options available to use with the cache tag helper. Don't use any of these because you will manually expire the cache whenever you add a new city or update an existing one. To achieve that, simply add the following line to both the `CreateAsync` and `UpdateAsync` methods within the `CityService`, just before their return statements:

```
_cache.Remove(nameof(GetCityNamesAsync));
```

Now you can use this new method within the City page. You will replace the existing code in the `OnGetAsync` method, so it can perform a lookup against the data returned by the new method. If the city name passed in the route exists, fetch its details from the database. If not, return a